# プログラミングワークショップ 2

青森大学ソフトウェア情報学部

橋本恭能

haship@aomori-u.ac.jp

# 講義資料について

- この講義で使う資料やプリントは以下のWebページから閲覧できます
- ブラウザを開き、以下のアドレスを入力してください

URL http://haship.github.io

- Webページ上部の【講義資料】をクリック
- 表から【プログラミングワークショップ２】をクリック
- 目次ページをブックマークする

# 目次

- テーマ：ブロック崩し
- 例題：ボールが左右に反射する
- 課題：ボールが上下に反射する

# ブロック崩し

- 後期の前半はゲームのブロック崩しをテーマに文法の振り返りをしていく
  - 関数（第8章）
  - オブジェクト(第9章)
  - 配列(第10章)

- 後期の後半はソフトウェア開発

# ボールが壁に反射する

- EX_08_05を参考に、ボールが左右で反射するプログラムを作成

# プログラム例 EX_08_05

```
int radius = 40;            // ボールの半径
float x = 110;              // ボールのx座標（初期値110）
float speed = 0.5;          // ボールの速度（初期値0.5）
int direction = 1;          // ボールの進行方向（初期値1、-1:左方向、1:右方向）

void setup() {
  size(240, 120);
  ellipseMode(RADIUS);      // 半径を指定して円を描画する設定に変更
}

void draw() {
  background(0);
  x += speed * direction;                       // ボールが移動する
  if ((x > width-radius) || (x < radius)) {     // 右壁か左壁に衝突する場合
    direction = -direction;                     // 進行方向を反転する
  }
  ellipse(x, 60, radius, radius);               // ボールを描く
}
```

# 関数を使って書き直す

- draw()関数内の処理を以下の関数に置き換える
  - `void move()` ボールが移動する
  - `void flipXdir()` ボールをx方向に反転する
  - `boolean isHitRightWall()` ボールが右壁に衝突する
  - `boolean isHitLeftWall()` ボールが左壁に衝突する
  - `void display()` ボールを描く

# ボールが移動する

変更前

```
void draw() {
  background(0);
  x += speed * direction;
  if ((x > width-radius) ||
                      (x < radius)) {
    direction = -direction;
  }
  ellipse(x, 60, radius, radius);
}
```

変更後

```
void draw() {
  background(0);
  move();
  if ((x > width-radius) ||
                      (x < radius)) {
    direction = -direction;
  }
  ellipse(x, 60, radius, radius);
}

// ボールが移動する
void move(){
  x += speed * direction;
}
```

関数の呼出

関数の定義

# ボールをx方向に反転する

変更前

```
void draw() {
  background(0);
  move();
  if ((x > width-radius) ||
                      (x < radius)) {
    direction = -direction;
  }
  ellipse(x, 60, radius, radius);
}

// ボールが移動する
void move(){
  x += speed * direction;
}
```

変更後

```
void draw() {
  background(0);
  move();
  if ((x > width-radius) ||
                      (x < radius)) {
    flipXdir();
  }
  ellipse(x, 60, radius, radius);
}

// ボールが移動する
void move(){
  x += speed * direction;
}
// ボールの進行方向を反転する
void flipXdir(){
  direction = -direction;
}
```

関数の呼出

関数の定義

# ボールを描く

変更前

```
void draw() {
  background(0);
  move();
  if ((x > width-radius) ||
                      (x < radius)) {
    flipXdir();
  }
  ellipse(x, 60, radius, radius);
}

// ボールが移動する
void move(){
  x += speed * direction;
}
// ボールの進行方向を反転する
void flipXdir(){
  direction = -direction;
}
```

変更後

```
void draw() {
  background(0);
  move();
  if ((x > width-radius) ||
                      (x < radius)) {
    flipXdir();
  }
  display();
}

// ボールが移動する
void move(){
  x += speed * direction;
}
// ボールの進行方向を反転する
void flipXdir(){
  direction = -direction;
}
//ボールを描く
void display(){
  ellipse(x, 60, radius, radius);
}
```

関数の呼出

関数の定義

# ボールが右壁か左壁に衝突する場合

- 条件が2つあるので分ける
  - ボールが右壁に衝突するかどうか
  - ボールが左壁に衝突するかどうか
- 条件を満たしたらtrueを返し、それ以外はfalseを返す関数を作る
- 条件ごとにboolean型の関数
  - `boolean isHitRightWall()`
  - `boolean isHitLeftWall()`

# 右壁に衝突したかどうか

変更前

```
void draw() {
  background(0);
  move();
  if ((x > width-radius) ||
                        (x < radius)) {
    flipXdir();
  }
  display();
}

// ボールが移動する（省略）
// ボールの進行方向を反転する（省略）
// ボールを描く（省略）
```

変更後

```
void draw() {
  background(0);
  move();
  if ( isHitRightWall() ||
                        (x < radius)) {
    flipXdir();
  }
  display();
}

// ボールが移動する（省略）
// ボールの進行方向を反転する（省略）
// ボールを描く（省略）
// 右壁に衝突したかどうか
boolean isHitRightWall(){
  if(x > width - radius){
    return true;
  } else {
    return false;
  }
}
```

関数の呼出

関数の定義

# 左壁に衝突したかどうか

```
void draw() {
  background(0);
  move();
  if ( isHitRightWall() ||
                        (x < radius)) {
    flipXdir();
  }
  display();
}

// ボールが移動する（省略）
// ボールの進行方向を反転する（省略）
// ボールを描く（省略）
// 右壁に衝突したかどうか（省略）
```

```
void draw() {
  background(0);
  move();
  if ( isHitRightWall() ||
                        isHitLeftWall() ) {
    flipXdir();
  }
  display();
}
// ボールが移動する（省略）
// ボールの進行方向を反転する（省略）
// ボールを描く（省略）
// 右壁に衝突したかどうか（省略）
// 左壁に衝突したかどうか
boolean isHitLeftWall(){
  if(x < radius){
    return true;
  } else {
    return false;
  }
}
```

関数の呼出

関数の定義

# プログラム例 変更後

```
int radius = 40;            // ボールの半径
float x = 110;              // ボールのx座標（初期値110）
float speed = 0.5;          // ボールの速度（初期値0.5）
int direction = 1;          // ボールの進行方向（初期値1、-1:左方向、1:右方向）

void setup() {
  size(240, 120);
  ellipseMode(RADIUS);      // 半径を指定して円を描画する設定に変更
}

void draw() {
  background(0);
  move();                                          //ボールが移動する
  if ( isHitRightWall() || isHitLeftWall() ) {  //右壁か左壁に衝突したら
    flipXdir();                                    //x方向に反転する
  }
  display();                                       //ボールを描く
}
```

# プログラム例 変更後

```
// ボールが移動する
void move(){
  x += speed * direction;
}

// ボールをx方向に反転する
void flipXdir(){
  direction = -direction;
}

//ボールを描く
void display(){
  ellipse(x, 60, radius, radius);
}
```

```
// 右壁に衝突したかどうか
boolean isHitRightWall(){
  if(x > width – radius){
    return true;
  } else {
    return false;
  }
}

// 左壁に衝突したかどうか
boolean isHitLeftWall(){
  if(x < radius){
    return true;
  } else {
    return false;
  }
}
```

# 課題 Kadai_01

- 例題を参考に、ボールが上下で反射するプログラムの一部を関数化する

| | |
|---|---|
| ボールが移動する | `void move()` |
| ボールが下壁に衝突するかどうか | `boolean isHitBottomWall()` |
| ボールが上壁に衝突するかどうか | `boolean isHitTopWall()` |
| ボールをY方向に反転する | `void flipYdir()` |
| ボールを描く | `void display()` |

- pw1の課題 Kadai_08_01を基にしています

# プログラム変更前

```
int radius = 40;            // ボールの半径
float y = 60;               // ボールのy座標（初期値60）
float speed = 0.5;          // ボールの速度（初期値0.5）
int direction = 1;          // ボールの進行方向（初期値1、-1:上方向、1:下方向）

void setup() {
  size(240, 120);
  ellipseMode(RADIUS);   // 半径を指定して円を描画する設定に変更
}

void draw() {
  background(0);
  y += speed * direction;                    // ボールが移動する
  if ((y > height-radius) || (y < radius)) {  // 下壁か上壁に衝突する場合
    direction = -direction;                  // 進行方向を反転する
  }
  ellipse(110, y, radius, radius);           // ボールを描く
}
```

# Kadai_01

```
int radius = 40;              // ボールの半径
float y = 60;                 // ボールのy座標（初期値60）
float speed = 0.5;            // ボールの速度（初期値0.5）
int direction = 1;            // ボールの進行方向（初期値1、0:左方向、1:右方向）

void setup() {
  size(240, 120);
  ellipseMode(RADIUS);        // 半径を指定して円を描画する設定に変更
}

void draw() {
  background(0);
                                          // ボールが移動する
  if (                  ||                ) {  // 下壁か上壁に衝突する場合
                                          // 進行方向を反転する
  }
                                          // ボールを描く
}
```

# Kadai_01

```
// ボールが移動する
void move(){

}

// ボールをy方向に反転する
void flipYdir(){

}

//ボールを描く
void display(){

}
```

```
// 下壁に衝突したかどうか
boolean isHitbottomWall(){

}

// 上壁に衝突したかどうか
boolean isHitTopWall(){

}
```